



Web programming: Module I

▼ Introduction to the Web

▼ The World Wide Web(WWW)

The **World Wide Web** is abbreviated as WWW and is commonly known as the Web. The WWW was initiated by CERN (European Library for Nuclear Research) in 1989.

WWW can be defined as the collection of different websites around the world, containing different information shared via local servers(or computers).

Working of WWW:

The World Wide Web is based on several different technologies: Web browsers, Hypertext Markup Language (HTML), and Hypertext Transfer Protocol (HTTP).

A Web browser is used to access web pages. Web browsers are programs that display text, data, pictures, animation, and video on the Internet. Hyperlinked resources on the World Wide Web can be accessed using software interfaces provided by Web browsers. Initially, Web browsers were used only for surfing the Web but now they have become more universal. Web browsers can be used for several tasks including conducting searches, mailing, transferring files, and much more. Some of the commonly used browsers are Internet Explorer, Opera Mini, and Google Chrome.

Features of WWW:

- Hyper-Text Information System
- Cross-Platform
- Distributed
- Open Standards and Open Source
- Uses Web Browsers to provide a single interface for many services
- Dynamic, Interactive, and Evolving.
- “Web 2.0”

Components of the Web: There are 3 components of the web:

1. **Uniform Resource Locator (URL):** serves as a system for resources on the web.
2. **HyperText Transfer Protocol (HTTP):** specifies communication of browser and server.

3. **Hyper Text Markup Language (HTML):** defines the structure, organization, and content of a web page

▼ Web Basics

▼ Hyperlinks

Hyperlinks in HTML are used to create clickable links that allow users to navigate between different web pages or sections within the same page. In HTML, hyperlinks are created using the `<a>` (anchor) element. The basic syntax for creating a hyperlink is as follows:

```
<a href="url">Link text</a>
```

- The `<a>` element is the anchor element, which defines a hyperlink.
- The `href` attribute specifies the destination URL that the link should point to. It can be an absolute URL (e.g., "<https://www.example.com>") or a relative URL (e.g., "page.html").
- The link text is the visible text that users can click on. It goes between the opening and closing `<a>` tags.

Eg:

```
<a href="https://www.example.com">Visit Example Website</a>
```

In this example, the link text is "Visit Example Website," and when users click on it, they will be directed to the URL "<https://www.example.com>".

▼ URIs & URLs

<https://youtu.be/vpYct2npKD8>

URI, URL, and URN are all related terms used in the context of identifying and locating resources on the internet. Here's what they stand for:

URI: URI stands for Uniform Resource Identifier. It is a string of characters used to identify and locate a resource. A URI can be further classified into two types: URLs and URNs.

URL: URL stands for Uniform Resource Locator. It is a specific type of URI that provides the means to access a resource on the internet. A URL typically consists of several components, including the protocol (such as "http" or "https"), the domain name or IP address of the server, and

the path to the resource. For example, "<https://www.example.com/index.html>" is a URL that specifies an HTTPS protocol, the domain name "www.example.com," and the file "index.html" located on that server.

URN: URN stands for Uniform Resource Name. It is another type of URI that is used to identify a resource by its name or identifier rather than by its location. URNs are persistent and unique, allowing resources to be identified even if they are moved or renamed. However, unlike URLs, URNs do not provide a means to directly access the resource. An example of a URN is "urn:isbn:0451450523," which identifies a book by its ISBN number.

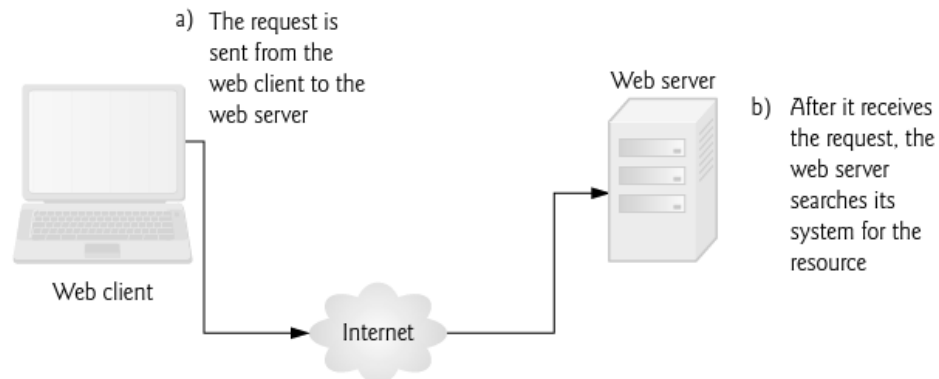
▼ Parts of URL

A URL (Uniform Resource Locator) is a string of characters that provides the address of a resource on the internet. Let's break down the components of a URL based on the information you provided:

1. **Protocol:** The protocol is indicated at the beginning of the URL and specifies the method to be used for accessing the resource. In this case, "http://" indicates the use of the HTTP (HyperText Transfer Protocol) for retrieving the resource. Other protocols include "https://" for secure HTTP, "ftp://" for File Transfer Protocol, and so on.
2. **Hostname:** The hostname identifies the web server computer on which the resource is located. In your example, the hostname is "www.deitel.com." The hostname is a part of the fully qualified domain name (FQDN) and is used to uniquely identify the server. The FQDN typically consists of a combination of the hostname and the domain name (e.g., "www" is the hostname, and "deitel.com" is the domain name).
3. **IP Address:** The hostname is translated into an IP address, which is a numerical value that uniquely identifies the server on the internet. The translation from hostname to IP address is performed by DNS (Domain Name System) servers. These servers maintain a database of hostnames and their corresponding IP addresses.
4. **Resource Location:** The remaining part of the URL ("/books/downloads.html") specifies the location and name of the resource on the web server. In this example, "/books" represents the location or directory on the server's file system where the resource is located, and "downloads.html" is the name of the resource file. It could be an HTML page, an image, a document, or any other type of resource.

▼ Making a Request and Receiving a Response

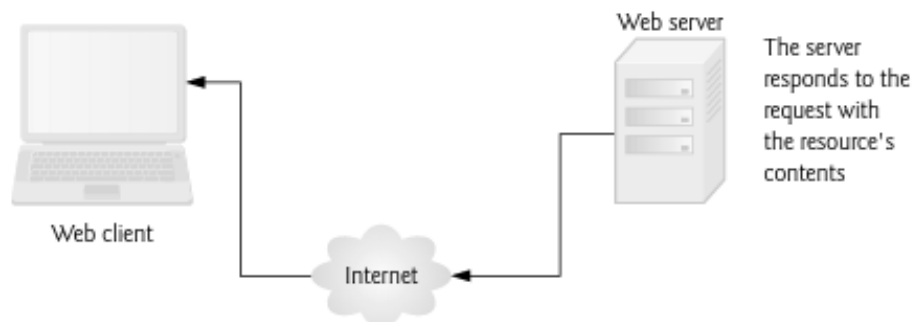
When given a web page URL, a web browser uses HTTP to request the web page found at that address.



The web browser sends an HTTP request to the server. The request (in its simplest form) is:

```
GET /books/downloads.html HTTP/1.1
```

The word **GET** is an HTTP method indicating that the client wishes to obtain a resource from the server. The remainder of the request provides the path name of the resource (e.g., an HTML5 document) and the protocol's name and version number (HTTP/1.1). The client's request also contains some required and optional headers. Any server that understands HTTP (version 1.1) can translate this request and respond appropriately.



The server first sends a line of text that indicates the HTTP version, followed by a numeric code and a phrase describing the status of the transaction.

Eg:

```
HTTP/1.1 200 OK
```

indicates success, whereas

```
HTTP/1.1 404 Not found
```

informs the client that the web server could not locate the requested resource.

▼ HTTP Headers

The server sends one or more HTTP headers, which provide additional information about the data that will be sent. In this case, the server is sending an HTML5 text document, so one HTTP header for this example would read:

```
Content-type: text/html
```

The information provided in this header specifies the **Multipurpose Internet Mail Extensions (MIME) type** of content that the server is transmitting to the browser. The MIME standard specifies data formats, which programs can use to interpret data correctly.

Eg:

The MIME type text/plain indicates that the sent information is text that can be displayed directly. Similarly, the MIME type image/jpeg indicates that the content is a JPEG image.

▼ HTTP get and post Request

1. GET request:

- The GET method is used to request a resource from the server.
- When a client sends a GET request, it is asking the server to retrieve a specific resource identified by a URL (Uniform Resource Locator).
- The parameters and data associated with a GET request are appended to the URL as query parameters.
- GET requests are typically used for retrieving data, and they are considered safe and idempotent, meaning they should not have any side effects on the server.

2. POST request:

- The POST method is used to send data to the server to create or update a resource.
- Unlike GET requests, the data sent in a POST request is not appended to the URL but is included in the body of the request.
- POST requests are commonly used for submitting forms, uploading files, or performing any action that modifies data on the server.

- POST requests are not idempotent, meaning multiple identical requests may have different effects on the server.

▼ Client-side Caching

Client-side caching is a mechanism employed by web browsers to store and reuse recently viewed web pages, resulting in quicker page reloading. When a browser retrieves a web page, it can cache the content locally and refer to it for subsequent requests, avoiding the need to fetch the data again from the server. The browser determines the freshness of the cached content based on the HTTP response headers, such as "Cache-Control" or "Expires," which specify the duration the content remains fresh.

▼ Client-side scripting v/s server-side scripting

What is a Script?

In computer programming, a script is a program or sequence of instructions that is interpreted or carried out by another program rather than by the computer processor (as a compiled program is).

Client-Side Scripting	Server-Side Scripting
Runs on the user's computer	Runs on the server
Code is visible to the user	Code is not visible to the user
Can be disabled by the user	Cannot be disabled by the user
Faster response time	Slower response time
Limited access to server resources	Full access to server resources
Limited security	Strong security

▼ Hyper Text Transfer Protocol (HTTP)

HTTP is a stateless protocol, meaning that the server does not keep any data (state) between two requests. This makes it efficient for handling large numbers of concurrent connections. HTTP is also a text-based protocol, which means that the messages that are sent between the client and the server are in plain text. This makes it easy to debug and troubleshoot HTTP problems.

HTTP is a request-response protocol. The client sends a request to the server, and the server sends a response back to the client. The request and response messages are made up of a series of headers and a body. The headers contain information about the request or response, such as the type of request, the length of the body, and the encoding of the body. The body contains the actual data that is being transferred.

▼ Common Gateway Interface (CGI)

Common Gateway Interface (CGI) is a standard that enables communication between web servers and external databases or information sources. It acts as middle-ware, allowing web servers to interact with applications that process data and send back responses.

CGI is often used to process input information from the user and produce the appropriate output. An example of a CGI program is one implementing a wiki. If the user agent requests the name of an entry, the Web server executes the CGI program. The CGI program retrieves the source of that entry's page (if one exists), transforms it into HTML, and prints the result. The Web server receives the output from the CGI program and transmits it to the user agent. Then if the user agent clicks the "Edit page" button, the CGI program populates an HTML textarea or other editing control with the page's contents.

▼ Introduction to HTML

▼ Origins and Evolution of HTML

HTML Origins

- HTML was created by Tim Berners-Lee in 1991.
- It was originally called "HyperText Markup Language".
- The goal was to create a way to share information across different computers.
- HTML was originally designed to be used with the NeXTSTEP operating system.

HTML Evolution

- HTML has evolved over time to add new features and improve the way it works.
- Some of the major changes include:
 - HTML 2.0 (1994): Added support for tables, forms, and images.
 - HTML 3.2 (1996): Added support for multimedia, frames, and style sheets.
 - HTML 4.01 (1999): Added support for internationalization, accessibility, and improved semantics.
 - XHTML (2000): A stricter version of HTML that is based on XML.
 - HTML5 (2014): The latest major version of HTML. It adds support for new features such as video, audio, and canvas elements.

HTML Today

- HTML is the most widely used markup language in the world.
- It is used to create web pages that are displayed by web browsers.
- HTML is a powerful tool that can be used to create a wide variety of web pages, from simple text pages to complex interactive applications.

▼ Basic Syntax of HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>My First HTML Document</title>
  </head>
  <body>
    <h1>This is a heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

This document will create a web page with a heading and a paragraph. The heading will be displayed in large, bold text, and the paragraph will be displayed in normal text.

This is a heading

This is a paragraph.

▼ Standard HTML Document Structure Basic Text Markup

```
<!DOCTYPE html>
<html>
  <head>
    <title>Document Title</title>
  </head>
  <body>
    <!-- Page Content -->
  </body>
</html>
```

The `<!DOCTYPE html>` declaration tells the browser that this document is an HTML5 document.

The `<html>` element is the root element of an HTML document. It contains the `<head>` and `<body>` elements.

The `<head>` element contains meta-information about the document, such as the title and the character set.

The `<title>` element specifies the title of the document. It is displayed in the browser's title bar.

The `<body>` element contains the main content of the document. It can contain headings, paragraphs, images, tables, forms, and other elements.

▼ Lists

unordered List

An ordered list represents a list of items in a specific order, typically displayed with numbers or letters. Each list item is wrapped in `<li>` tags.

Example:

```
<ul>
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>
```

- Coffee
- Tea
- Milk

ordered List

An unordered list represents a list of items without any particular order, typically displayed with bullet points. Each list item is wrapped in `<li>` tags.

Example:

```
<ol>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ol>
```

1. Coffee
2. Tea
3. Milk

Definition List

A definition list represents a list of terms and their corresponding definitions. Each term is wrapped in `<dt>` tags, and each definition is wrapped in `<dd>` tags.

Example:

```

<dl>
  <dt>Term 1</dt>
  <dd>Definition 1</dd>
  <dt>Term 2</dt>
  <dd>Definition 2</dd>
  <dt>Term 3</dt>
  <dd>Definition 3</dd>
</dl>

```

```

Term 1
  Definition 1
Term 2
  Definition 2
Term 3
  Definition 3

```

▼ Tables

In HTML, tables are used to display tabular data in a structured format. Tables consist of rows (`<tr>`) and columns (`<td>` or `<th>`). The `<td>` element represents a standard cell within a table, while the `<th>` element represents a header cell. Here's an example of how to create a basic HTML table:

```

<table>
  <tr>
    <th>header 1</th>
    <th>header 2</th>
    <th>header 3</th>
  </tr>
  <tr>
    <td>Data 1</td>
    <td>Data 2</td>
    <td>Data 3</td>
  </tr>

  <tr>
    <td>Data 4</td>
    <td>Data 5</td>
    <td>Data 6</td>
  </tr>
</table>

```

```

header 1 header 2 header 3
Data 1   Data 2   Data 3
Data 1   Data 2   Data 3

```

▼ images

it is possible to add images into a webpage

```
<!DOCTYPE html>

<html>
  <head>
    <title>Ima400ge </title>
    
  </head>
</html>
```



▼ Hypertext Links

Hypertext is a method of organizing and presenting information in which text is linked together through hyperlinks. It allows users to navigate between different pieces of text or information by clicking on links within a document or web page

```
<!DOCTYPE html>
<html>
  <head>
    <title>Hypertext Links</title>
  </head>
  <body>
    <h1>HyperLinks</h1>
    <a href="http://itanimulli.com">click Here</a>
  </body>
</html>
```



HyperLinks

[click Here](#)

▼ Forms

Forms are used to collect user input and submit it to a server for processing. Forms typically contain input fields, checkboxes, radio buttons, dropdown menus, and buttons for submitting or resetting the form.

Eg:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Forms</title>
  </head>

  <body>
    <h1>Forms Example</h1>
    <form>
      <label for ="name">Name:</label>
      <input type="text" id="name" name="name" required>

      <label for ="email">Email:</label>
      <input type="text" id="email" name="email">

      <label for="message">Message:</label>
      <textarea id="message" name="message" rows="4" cols="30"></textarea>

    </form>
  </body>
</html>
```

output

Forms Example

Name: Email: Message:

- `<label>` tags associated with each input field, providing a description for the corresponding input.
- `<input>` tags with different types (`type="text"` , `type="email"`) for text input and email input fields.
- `<textarea>` tag for multi-line text input, which is useful for longer messages or comments.
- `<input>` tags with `type="submit"` and `type="reset"` for the submit and reset buttons, respectively.

The `name` attribute is used to identify each input field, and the `id` attribute is used to associate labels with their corresponding input elements.

When the form is submitted, the data is sent to the server for processing. The server-side code, typically written in a language like PHP, Python, or JavaScript, receives the form data and can perform actions like storing it in a database or sending an email.

Remember to adjust the `action` attribute of the `<form>` tag to the appropriate URL or script that will handle the form submission on your server.

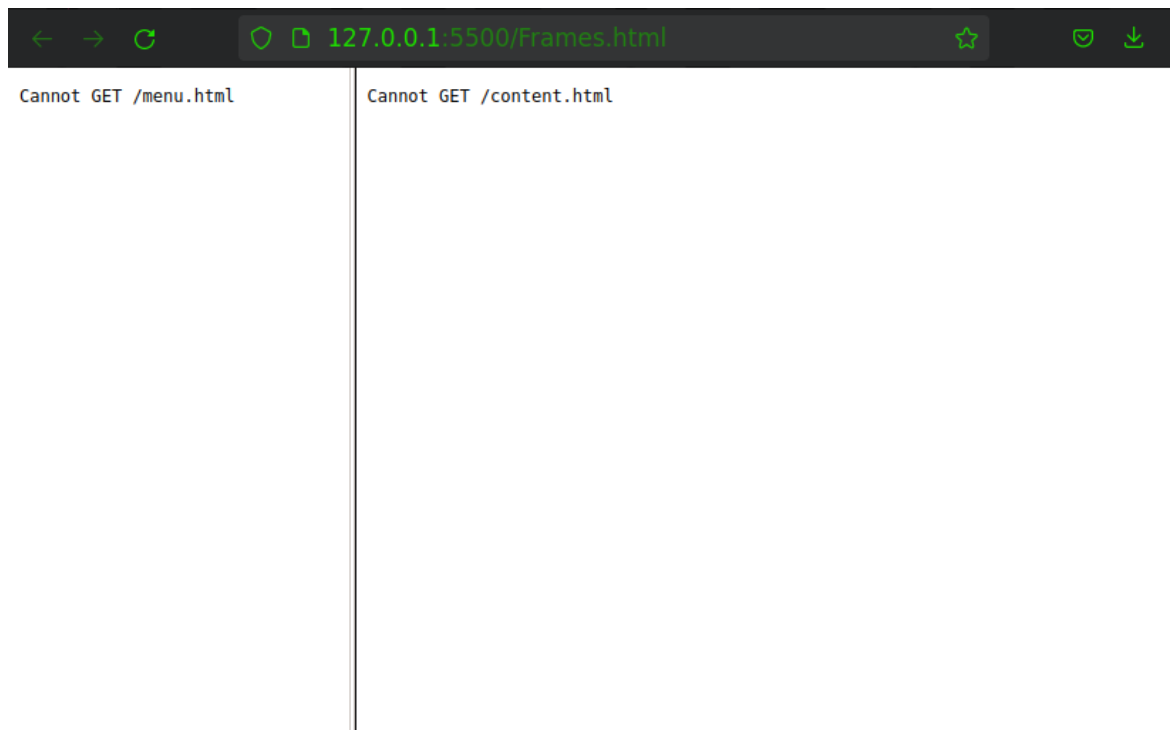
▼ Frames

Frames were an HTML feature that allowed web developers to divide a web page into multiple sections, each containing its own independent HTML document or webpage.

```
<!DOCTYPE html>
<html>
<head>
  <title> Frame Example </title>
</head>

<frameset cols="25%,75%">
  <frame src="menu.html" name="menu">
  <frame src="content.html" name="content">
</frameset>

</html>
```



2

Web programming: Module II

▼ Advanced HTML

▼ HTML DOM

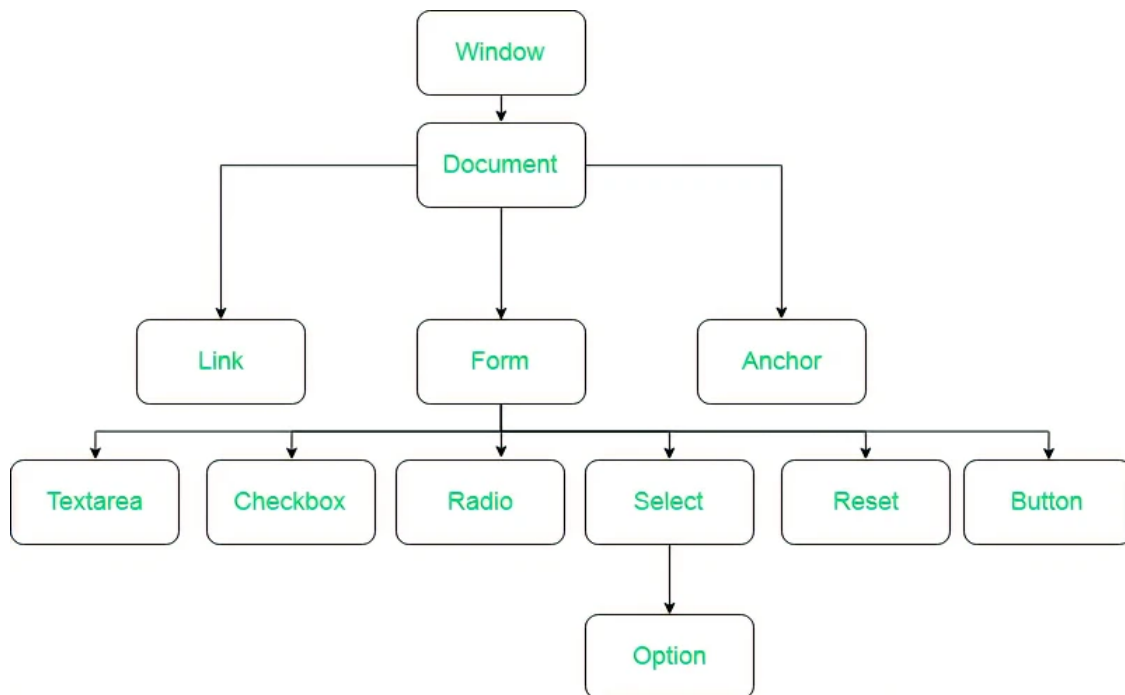
The Document Object Model is a *programming interface* for HTML(Hyper Text Markup Language) and XML(Extensible markup language) documents. It defines the **logical structure** of documents and the way a document is accessed and manipulated.

Note: It is called a Logical structure because DOM doesn't specify any relationship between objects.

Why DOM is required?

HTML is used to **structure** the web pages and Javascript is used to add **behavior** to our web pages. When an HTML file is loaded into the browser, the javascript can not understand the HTML document directly. So, a corresponding document is created(DOM). **DOM is basically the representation of the same HTML document but in a different format with the use of objects.** Javascript interprets DOM easily i.e. javascript can not understand the tags(<h1>H</h1>) in HTML document but can understand object h1 in DOM. Now, Javascript can access each of the objects (h1, p, etc) by using different functions.

Structure of DOM: DOM can be thought of as a Tree or Forest(more than one tree). The term **structure model** is sometimes used to describe the tree-like representation of a document. Each branch of the tree ends in a node, and each node contains objects. Event listeners can be added to nodes and triggered on an occurrence of a given event. One important property of DOM structure models is **structural isomorphism**: if any two DOM implementations are used to create a representation of the same document, they will create the same structure model, with precisely the same objects and relationships.



Properties of DOM

- **Window Object:** Window Object is an object of the browser that is always at the top of the hierarchy. It is like an API that is used to set and access all the properties and methods of the browser. It is automatically created by the browser.
- **Document object:** When an HTML document is loaded into a window, it becomes a document object. The 'document' object has various properties that refer to other objects which allow access to and modification of the content of the web page. If there is a need to access any element in an HTML page, we always start with accessing the 'document' object. Document object is property of window object.
- **Form Object:** It is represented by *form* tags.
- **Link Object:** It is represented by *link* tags.
- **Anchor Object:** It is represented by *a href* tags.
- **Form Control Elements:** Form can have many control elements such as text fields, buttons, radio buttons, checkboxes, etc.

▼ Difference between HTML and HTML5

HTML	HTML5
It didn't support audio and video without the use of Flash player support.	It supports audio and video controls with the use of <audio> and <video> tags.
It uses cookies to store temporary data.	It uses SQL databases and application cache to store offline data.
Does not allow JavaScript to run in the browser.	Allows JavaScript to run in the background. This is possible due to JS Web worker API in HTML5.
Vector graphics is possible in	Vector graphics is additionally an

HTML with the help of various technologies such as VML, Silver-light, Flash, etc.	integral part of HTML5 like SVG and Canvas.
It does not allow drag-and-drop effects.	It allows drag-and-drop effects.
Not possible to draw shapes like circles, rectangles, triangles, etc.	HTML5 allows you to draw shapes like circles, rectangle, triangles, etc.
It works with all old browsers.	It is supported by all new browsers like Firefox, Mozilla, Chrome, Safari, etc.
<HTML>, <Body> , and <Head> tags are mandatory while writing a HTML code.	These tags can be omitted while writing HTML code.
Older versions of HTML are less mobile-friendly.	HTML5 language is more mobile-friendly.
The doctype declaration is too long and complicated.	Doctype declaration is quite simple and easy.
Elements like nav, and header were not present.	A new element for web structure like nav, header, footer, etc.
Character encoding is long and complicated.	Character encoding is simple and easy.
It is almost impossible to get the true Geolocation of the user with the help of a browser.	One can track the geo location of a user easily by using JS Geolocation API.
It can not handle inaccurate syntax.	It is capable of handling inaccurate syntax.
Being an older version, it is not fast, flexible , and efficient as compared to HTML5.	It is efficient, flexible and faster in comparison to HTML.
Attributes like charset, async, and ping are absent in HTML.	Attributes of charset, async, and ping are a part of HTML 5.

▼ Semantic elements in HTML5

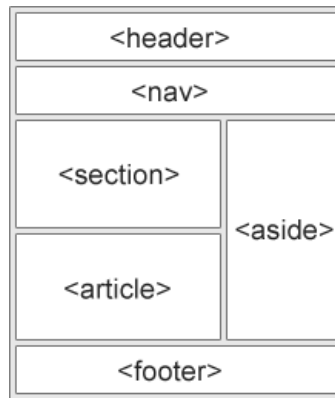
What are Semantic Elements?

A semantic element clearly describes its meaning to both the browser and the developer.

Examples of **non-semantic** elements: `<div>` and `<span>` - Tells nothing about its content.

Examples of **semantic** elements: `<form>`, `<table>`, and `<article>` - Clearly defines its content.

```
- <article>
- <aside>
- <details>
- <figcaption>
- <figure>
- <footer>
- <header>
- <main>
- <mark>
- <nav>
- <section>
- <summary>
- <time>
```



HTML <section> Element

The `<section>` element defines a section in a document.

According to W3C's HTML documentation: "A section is a thematic grouping of content, typically with a heading."

Examples of where a `<section>` element can be used:

- Chapters
- Introduction
- News items
- Contact information

A web page could normally be split into sections for introduction, content, and contact information.

Eg:

```
<!DOCTYPE html>
<html>
<body>

<section>
  <h1>WWF</h1>
  <p>The World Wide Fund for Nature (WWF) is an international organization working on issues regarding the conservation, research and restoration of the environment, formerly named the World Wildlife Fund. WWF was founded in 1961.</p>
</section>

<section>
  <h1>WWF's Panda symbol</h1>
  <p>The Panda has become the symbol of WWF. The well-known panda logo of WWF originated from a panda named Chi Chi that was transferred from the Beijing Zoo to the London Zoo in the same year of the establishment of WWF.</p>
</section>

</body>
</html>
```

WWF

The World Wide Fund for Nature (WWF) is an international organization working on issues regarding the conservation, research and restoration of the environment, formerly named the World Wildlife Fund. WWF was founded in 1961.

WWF's Panda symbol

The Panda has become the symbol of WWF. The well-known panda logo of WWF originated from a panda named Chi Chi that was transferred from the Beijing Zoo to the London Zoo in the same year of the establishment of WWF.

HTML <article> Element

The `<article>` element specifies independent, self-contained content.

An article should make sense on its own, and it should be possible to distribute it independently from the rest of the web site.

Examples of where the `<article>` element can be used:

- Forum posts
- Blog posts
- User comments
- Product cards
- Newspaper articles

Eg:

```
<!DOCTYPE html>
<html>
<body>

<h1>The article element</h1>

<article>
  <h2>Google Chrome</h2>
  <p>Google Chrome is a web browser developed by Google, released in 2008. Chrome is the world's most popular web browser today!</p>
</article>

<article>
  <h2>Mozilla Firefox</h2>
  <p>Mozilla Firefox is an open-source web browser developed by Mozilla. Firefox has been the second most popular web browser since January, 2018.</p>
</article>

<article>
  <h2>Microsoft Edge</h2>
  <p>Microsoft Edge is a web browser developed by Microsoft, released in 2015. Microsoft Edge replaced Internet Explorer.</p>
</article>

</body>
</html>
```

The article element

Google Chrome

Google Chrome is a web browser developed by Google, released in 2008. Chrome is the world's most popular web browser today!

Mozilla Firefox

Mozilla Firefox is an open-source web browser developed by Mozilla. Firefox has been the second most popular web browser since January, 2018.

Microsoft Edge

Microsoft Edge is a web browser developed by Microsoft, released in 2015. Microsoft Edge replaced Internet Explorer.

Nesting `<article>` in `<section>` or Vice Versa?

The `<article>` element specifies independent, self-contained content.

The `<section>` element defines section in a document.

Can we use the definitions to decide how to nest those elements? No, we cannot!

So, you will find HTML pages with `<section>` elements containing `<article>` elements, and `<article>` elements containing `<section>` elements.

HTML <header> Element

The `<header>` element represents a container for introductory content or a set of navigational links.

A `<header>` element typically contains:

- one or more heading elements (`<h1>` - `<h6>`)
- logo or icon
- authorship information

Note: You can have several `<header>` elements in one HTML document

.However, `<header>` cannot be placed within a `<footer>`, `<address>` or another `<header>` element.

Eg:

```
<!DOCTYPE html>
<html>
<body>

<article>
  <header>
    <h1>What Does WWF Do?</h1>
    <p>WWF's mission:</p>
  </header>
  <p>WWF's mission is to stop the degradation of
our planet's natural environment, and build a
future in which humans live in harmony with nature.
</p>
</article>

</body>
</html>
```

What Does WWF Do?

WWF's mission:

WWF's mission is to stop the degradation of our planet's natural environment, and build a future in which humans live in harmony with nature.

HTML <footer> Element

The `<footer>` element defines a footer for a document or section.

A `<footer>` element typically contains:

- authorship information
- copyright information
- contact information
- sitemap
- back to top links
- related documents

You can have several `<footer>` elements in one document.

Eg:

```

<!DOCTYPE html>
<html>
<body>

<footer>
  <p>Author: Hege Refsnes</p>
  <p><a
href="mailto:hege@example.com">hege@example.com</a>
</p>
</footer>

</body>
</html>

```

Author: Hege Refsnes
hege@example.com

HTML <nav> Element

The `<nav>` element is used to define a section of a webpage that contains navigation links. It is typically used to create a navigation menu or a list of links that direct users to different sections or pages of a website. The `<nav>` element is part of the semantic HTML5 markup, which helps search engines and assistive technologies understand the structure and purpose of the content.

Eg:

```

<!DOCTYPE html>
<html>
<body>

<nav>
  <a href="/html/">HTML</a> |
  <a href="/css/">CSS</a> |
  <a href="/js/">JavaScript</a> |
  <a href="/jquery/">jQuery</a>
</nav>

</body>
</html>

```

[HTML](#) | [CSS](#) | [JavaScript](#) | [jQuery](#)

HTML <aside> Element

The `<aside>` element is used to mark content that is tangentially related to the main content of a webpage. It typically represents information that is considered separate from the main content but still related or of secondary importance. The content within an `<aside>` element can be considered as a sidebar, a pull quote, advertising, or any other content that is not central to the main content but provides additional context or supplementary information.

Eg:

```

<article>
  <h1>Main Article</h1>
  <p>This is the main content of the article.</p>

  <aside>
    <h2>Related Information</h2>
    <p>This is some additional information related
to the main content.</p>
  </aside>
</article>

```

Main Article

This is the main content of the article.

Related Information

This is some additional information related to the main content.

▼ Basic HTML5 tags

▼ Article Tag

The `<article>` tag specifies independent, self-contained content.

An article should make sense on its own and it should be possible to distribute it independently from the rest of the site.

Potential sources for the `<article>` element:

- Forum post
- Blog post

```

<!DOCTYPE html>
<html>
<body>

<h1>The article element</h1>

<article>
  <h2>Google Chrome</h2>
  <p>Google Chrome is a web browser developed by
Google, released in 2008. Chrome is the world's
most popular web browser today!</p>
</article>

<article>
  <h2>Mozilla Firefox</h2>
  <p>Mozilla Firefox is an open-source web browser
developed by Mozilla. Firefox has been the second
most popular web browser since January, 2018.</p>
</article>

<article>
  <h2>Microsoft Edge</h2>
  <p>Microsoft Edge is a web browser developed by
Microsoft, released in 2015. Microsoft Edge
replaced Internet Explorer.</p>
</article>

</body>
</html>

```

The article element

Google Chrome

Google Chrome is a web browser developed by Google, released in 2008. Chrome is the world's most popular web browser today!

Mozilla Firefox

Mozilla Firefox is an open-source web browser developed by Mozilla. Firefox has been the second most popular web browser since January, 2018.

Microsoft Edge

Microsoft Edge is a web browser developed by Microsoft, released in 2015. Microsoft Edge replaced Internet Explorer.

▼ Figure tag

The `<figure>` tag specifies self-contained content, like illustrations, diagrams, photos, code listings, etc.

it is not necessary to use the `<figure>` tag to display an image. You can display an image in HTML using the `<img>` tag alone. The `<figure>` tag is used when you want to associate additional information, such as a caption, with the image.

```
<!DOCTYPE html>
<html>
<body>

<h1>The figure and figcaption element</h1>

<figure>
  
  <figcaption>Fig.1 - Trulli, Puglia, Italy.
</figcaption>
</figure>

</body>
</html>
```

The figure and figcaption element



Fig.1 - Trulli, Puglia, Italy.

▼ Header tag

The `<header>` tag is an HTML element used to define a header section of a document or a specific section within a document. It is typically used to contain introductory content, branding elements, navigation menus, or other top-level content on a webpage.

```
<!DOCTYPE html>
<html>
<head>
  <title>Simple Header Example</title>
</head>
<body>
  <header>
    <h1>Welcome to My Website</h1>
  </header>
</body>
</html>
```

Welcome to My Website

▼ Hgroup tag

The `<hgroup>` element is used to group multiple heading elements (`<h1>` to `<h6>`) as a single heading, typically for styling or presentation purposes. However, due to its limited browser support and lack of standardized behavior, it is generally recommended to use individual heading elements to represent the document structure instead.

```

<!DOCTYPE html>
<html>
<head>
  <title>Example with hgroup</title>
</head>
<body>
  <hgroup>
    <h1>Main Heading</h1>
    <h2>Subheading</h2>
  </hgroup>
</body>
</html>

```

Main Heading

Subheading

▼ Footer tag

The `<footer>` tag is an HTML element used to define the footer section of a document or a specific section within a document. It typically contains information about the document such as authorship, copyright notices, contact information, related links, or any other relevant information.

```

<!DOCTYPE html>
<html>
<head>
  <title>Simple Footer Example</title>
</head>
<body>
  <main>
    <!-- Main content of the webpage goes here -->
  </main>

  <footer>
    <p>This is the footer of the webpage.</p>
  </footer>
</body>
</html>

```

This is the footer of the webpage.

The `<footer>` element is commonly used to provide important information and navigation links at the bottom of a webpage. It helps to separate the footer content from the main content and header sections, providing a clear visual distinction.

▼ Navigation tag

The `<nav>` tag is an HTML element used to define a section of a document that contains navigation links. It is specifically intended for structural navigation blocks, such as menus, lists of links, or other types of navigation elements.

```

<!DOCTYPE html>
<html>
<body>

<h1>The nav element</h1>

<p>The nav element defines a set of navigation
links:</p>

<nav>
<a href="/html/">HTML</a> |
<a href="/css/">CSS</a> |
<a href="/js/">JavaScript</a> |
<a href="/python/">Python</a>
</nav>

</body>
</html>

```

The nav element

The nav element defines a set of navigation links:

[HTML](#) | [CSS](#) | [JavaScript](#) | [Python](#)

▼ Section Tag

The `<section>` tag is an HTML element used to define a section or a thematic grouping of content within a document. It is used to organize and group related content together, such as articles, blog posts, chapters, or any other meaningful section of a webpage.

```

<!DOCTYPE html>
<html>
<body>

<section>
  <h1>WWF</h1>
  <p>The World Wide Fund for Nature (WWF) is an
international organization working on issues
regarding the conservation, research and
restoration of the environment, formerly named the
World Wildlife Fund. WWF was founded in 1961.</p>
</section>

<section>
  <h1>WWF's Panda symbol</h1>
  <p>The Panda has become the symbol of WWF. The
well-known panda logo of WWF originated from a
panda named Chi Chi that was transferred from the
Beijing Zoo to the London Zoo in the same year of
the establishment of WWF.</p>
</section>

</body>
</html>

```

WWF

The World Wide Fund for Nature (WWF) is an international organization working on issues regarding the conservation, research and restoration of the environment, formerly named the World Wildlife Fund. WWF was founded in 1961.

WWF's Panda symbol

The Panda has become the symbol of WWF. The well-known panda logo of WWF originated from a panda named Chi Chi that was transferred from the Beijing Zoo to the London Zoo in the same year of the establishment of WWF.

▼ Aside tag

The `<aside>` tag is an HTML element used to mark content that is tangentially related to the main content of a webpage. It is typically used to enclose content such as sidebars, pull quotes, author information, related links, or any other content that is considered separate from the main content but still related or supportive.

```

<!DOCTYPE html>
<html>
<body>

<h1>The aside element</h1>

<p>My family and I visited The Epcot center this
summer. The weather was nice, and Epcot was
amazing! I had a great summer together with my
family!</p>

<aside>
  <h4>Epcot Center</h4>
  <p>Epcot is a theme park at Walt Disney World
Resort featuring exciting attractions,
international pavilions, award-winning fireworks
and seasonal special events.</p>
</aside>

</body>
</html>

```

The aside element

My family and I visited The Epcot center this summer. The weather was nice, and Epcot was amazing! I had a great summer together with my family!

Epcot Center

Epcot is a theme park at Walt Disney World Resort featuring exciting attractions, international pavilions, award-winning fireworks and seasonal special events.

▼ Address tag

The `<address>` tag defines the contact information for the author/owner of a document or an article.

The contact information can be an email address, URL, physical address, phone number, social media handle, etc.

The text in the `<address>` element usually renders in *italic*, and browsers will always add a line break before and after the `<address>` element.

```

<!DOCTYPE html>
<html>
<body>

<h1>The address element</h1>

<address>
  Written by <a
href="mailto:webmaster@example.com">Jon
Doe</a>.<br>
  Visit us at:<br>
  Example.com<br>
  Box 564, Disneyland<br>
  USA
</address>

</body>
</html>

```

The address element

Written by [Jon Doe](mailto:Jon.Doe).
Visit us at:
Example.com
Box 564, Disneyland
USA

▼ HTML5 Canvas

The HTML `<canvas>` element is used to draw graphics on a web page.

The graphic to the left is created with `<canvas>`. It shows four elements: a red rectangle, a gradient rectangle, a multicolor rectangle, and a multicolor text.



```

<!DOCTYPE html>
<html>
<body>

<canvas id="myCanvas" width="200" height="100"
style="border:1px solid #FF0000;">
Your browser does not support the HTML canvas
tag.
</canvas>

</body>
</html>

```



▼ Introduction to Cascading Style Sheet

what is css?

CSS (Cascading Style Sheets) is a styling language used to describe the presentation and formatting of a document written in HTML or XML. It is used to control the layout, colors, fonts, and other visual aspects of a web page.

CSS works by selecting HTML elements and applying specific styles to them. These styles can be defined directly within an HTML document using the `<style>` tag, or they can be stored in separate CSS files and linked to the HTML document using the `<link>` tag.

▼ Types of Style Sheet

▼ External Style sheets

An external style sheet is used to define the style for many HTML pages.

To use an external style sheet, add a link to it in the `<head>` section of each HTML page:

"styles.css":

```

body {
  background-color: powderblue;
}
h1 {
  color: blue;
}
p {
  color: red;
}

```

```

<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <h1>This is a heading</h1>
  <p>This is a paragraph.</p>

</body>
</html>

```

This is a heading

This is a paragraph.

▼ Internal Style sheets

An internal CSS is used to define a style for a single HTML page.

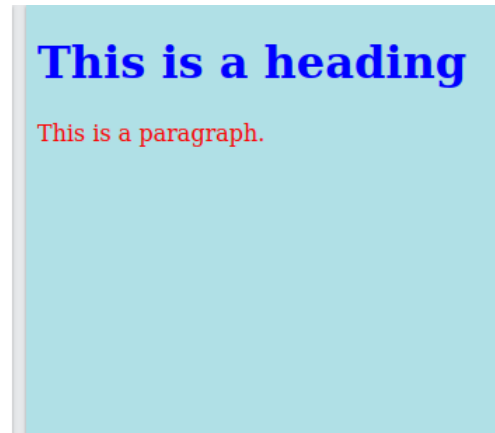
An internal CSS is defined in the `<head>` section of an HTML page, within a `<style>` element.

The following example sets the text color of ALL the `<h1>` elements (on that page) to blue, and the text color of ALL the `<p>` elements to red. In addition, the page will be displayed with a "powderblue" background color:

```
<!DOCTYPE html>
<html>
<head>
<style>
body {background-color: powderblue;}
h1 {color: blue;}
p {color: red;}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```



▼ Inline style sheets

An inline CSS is used to apply a unique style to a single HTML element.

An inline CSS uses the `style` attribute of an HTML element.

The following example sets the text color of the `<h1>` element to blue, and the text color of the `<p>` element to red:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="color:blue;">A Blue Heading</h1>

<p style="color:red;">A red paragraph.</p>

</body>
</html>
```



▼ Selector Forms

A CSS selector is the first part of a CSS Rule. It is a pattern of elements and other terms that tell the browser which HTML elements should be selected to have the CSS property values inside the rule applied to them. The element or elements which are selected by the selector are referred to as the *subject of the selector*.

Types of selectors :

▼ **Element Selectors**

The element selector selects the HTML element by name.

```
<!DOCTYPE html>
<html>
<head>

<style>
p {
  background-color: yellow;
}
</style>

</head>
<body>

<h1>Demo of the element selector</h1>

<div>
  <p>My name is Donald.</p>
  <p>I live in Duckburg.</p>
</div>

<p>My best friend is Mickey.</p>

</body>
```

Demo of the element selector

My name is Donald.

I live in Duckburg.

My best friend is Mickey.

▼ ID Selectors

The id selector selects the id attribute of an HTML element to select a specific element. An id is always unique within the page so it is chosen to select a single, unique element.

It is written with the hash character (#), followed by the id of the element.

```
<!DOCTYPE html>
<html>
<head>

<style>
#para1 {
  text-align: center;
  color: red;
}
</style>

</head>
<body>

<p id="para1">Hello World!</p>
<p>This paragraph is not affected by the style.</p>

</body>
</html>
```

Hello World!

This paragraph is not affected by the style.

▼ Class Selectors

The `.class` selector selects elements with a specific class attribute.

To select elements with a specific class, write a period (.) character, followed by the name of the class.

```

<!DOCTYPE html>
<html>
<head>

<style>
.intro {
  background-color: yellow;
}
</style>

</head>
<body>

<h1>Demo of the .class selector</h1>

<div class="intro">
  <p>My name is Donald.</p>
  <p>I live in Duckburg.</p>
</div>

<p>My best friend is Mickey.</p>

<p class="intro">My best friend is Mickey.</p>

</body>
</html>

```

Demo of the .class selector

My name is Donald.

I live in Duckburg.

My best friend is Mickey.

My best friend is Mickey.

▼ Universal Selectors

Select all elements, and set them a common style.

```

<!DOCTYPE html>
<html>
<head>

<style>
* {
  background-color: yellow;
}
</style>

</head>
<body>

<h1>Demo of the * selector</h1>

<div class="intro">
  <p id="firstname">The background color is
yellow.</p>
  <p id="hometown">I live in Duckburg.</p>
</div>

<p>My best friend is Mickey.</p>

</body>
</html>

```

Demo of the * selector

The background color is yellow.

I live in Duckburg.

My best friend is Mickey.

▼ Group Selectors

The grouping selector is used to select all the elements with the same style definitions. Grouping selector is used to minimize the code. Commas are used to separate each selector in grouping.

```

<!DOCTYPE html>
<html>
<head>

<style >
h1, h2, h3 {
    color: Maroon;
}
</style>

<h1>Main header</h1>

<h2>Header level 2</h2>

<h3>Header level 3</h3>

</body>
</html>

```

Main header

Header level 2

Header level 3

▼ Property-value Forms

A single line of CSS consists of two things, a **property** and a **value**. Properties are the things that you want to style, and values are the things that you want to apply to those properties. An example of a CSS declaration would be:

```
color: black;
```

The property here is `color` and the value is `black`.

Here are some commonly used property-value pairs in CSS:

1. `color: value;` - Specifies the text color of an element. The value can be a named color (e.g., `red`, `blue`), a hexadecimal color code (e.g., `#ff0000`), or an RGB/RGBA value (e.g., `rgb(255, 0, 0)`).
2. `font-size: value;` - Sets the size of the font used within an element. The value can be specified in pixels (`px`), points (`pt`), percentages (`%`), or other units.
3. `background-color: value;` - Defines the background color of an element. The value can be a named color, hexadecimal color code, or an RGB/RGBA value.
4. `width: value;` - Sets the width of an element. The value can be specified in pixels, percentages, or other units.
5. `height: value;` - Specifies the height of an element. The value can be defined in pixels, percentages, or other units.
6. `margin: value;` - Sets the margin (space) around an element. The value can be specified in pixels, percentages, or other units. You can use shorthand notations like `margin-top`, `margin-right`, `margin-bottom`, and `margin-left` to define individual margins.
7. `padding: value;` - Defines the padding (space) within an element. The value can be specified in pixels, percentages, or other units. Like margins, you can use shorthand notations (`padding-top`, `padding-right`, `padding-bottom`, and `padding-left`) to specify individual paddings.
8. `border: value;` - Specifies the border properties of an element. The value can include properties like border-width, border-style, and border-color. For example: `border: 1px solid black;`.

9. `text-align: value;` - Sets the alignment of the text within an element. The value can be `left`, `right`, `center`, or `justify`.

▼ Font properties

CSS Font property is used to control the look of texts. By the use of CSS font property you can change the text size, color, style and more. You have already studied how to make text bold or underlined. Here, you will also know how to resize your font using percentage.

Some of the font properties are:

▼ CSS Font Color

CSS font color is a standalone attribute in CSS although it seems that it is a part of CSS fonts. It is used to change the color of the text.

There are three different formats to define a color:

- By a color name
- By hexadecimal value
- By RGB

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
    font-size: 100%;
}
h1 { color: red; }
h2 { color: #9000A1; }
p { color: rgb(0, 220, 98); }
}
</style>
</head>
<body>
<h1>This is heading 1</h1>
<h2>This is heading 2</h2>
<p>This is a paragraph.</p>
</body>
</html>
```

This is heading 1

This is heading 2

This is a paragraph.

▼ CSS Font Size

In CSS, the font-size property is used to set or tweak the size of the font. It can have several values that can be absolute (e.g.- xx-small, medium, xx-large.) or relative (larger, smaller, %) or length (numbered- 12px, 1em, etc.)

```
font-size: xx-small | x-small | small | medium | large | x-large | xx-large ;
```

```

<!DOCTYPE html>
<html>
<head>
  <style>
    div.a {
      font-size: 20px;
    }

    div.b {
      font-size: large;
    }

    div.c {
      font-size: 200%;
    }

    div.d {
      font-size: small;
    }
  </style>
</head>
<body>
  <div class="a">This is 20px.</div>
  <div class="b">This is large.</div>
  <div class="c">This is 200%.</div>
  <div class="d">This is small</div>

</body>
</html>

```

This is 20px.
 This is large.
This is 200%.
This is small

▼ CSS Font-Weight

The weight of the font refers to the thickness of the font, i.e., how thick or thin the font will be on the webpage. The font-weight property of CSS is used to set the weight/thickness/boldness of the font.

It is either dependent on the specified weights of the browser or the available font faces in a font family. It can have absolute or relative, or numeric values like normal, bold, lighter, bolder, 100, 400. Though, We do not need to specify any unit while using the numeric values.

400 has the same weight as normal, and 700 have the same as bold.

```
font-weight: normal | bold | bolder | lighter | <number [1,1000]>;
```

```

<!DOCTYPE html>
<html>
<head>
  <style>
    p.normal {
      font-weight: normal;
    }

    p.light {
      font-weight: lighter;
    }

    p.thick {
      font-weight: bold;
    }

    p.thicker {
      font-weight: 600;
    }
  </style>
</head>
<body>
  <p class="normal">This line is normal</p>
  <p class="light">This line is lighter</p>
  <p class="thick">This line is bold</p>
  <p class="thicker">This line is of value
600</p>
</body>
</html>

```

This line is normal

This line is lighter

This line is bold

This line is of value 600

▼ CSS Font-Style

CSS font-style property is used to define the style of font for the text content of an element. Here, style refers to the variation in the typeface. It may be italic, oblique, or normal(default). Font-style property can be used to decorate and assign importance to a specific text.

```
font-style: normal | italic | oblique;
```

▼ CSS Font-Variant

The font-variant property in CSS allows you to transform the specific line of text or paragraph into **small caps** (capital letters that will be small in size as compared to normal capital letters).

It can also have values such as **normal** (by default), **initial**- This sets the property to the default value, and **inherit**- It takes the property's value of the parent element to the child element.

```
font-variant: normal | small-caps | initial | inherit;
```

```

<!DOCTYPE html>
<html>
<head>
  <style>
    p.normal {
      font-variant: normal;
    }

    p.small {
      font-variant: small-caps;
    }
  </style>
</head>
<body>

  <p class="normal">HTML in normal</p>
  <p class="small">HTML in small-caps</p>

</body>
</html>

```

Scaler Topics in normal

SCALER TOPICS IN SMALL-CAPS

▼ CSS Font-Family

A font family is a set of fonts having similar typefaces and common designs. It is important to choose the right font family because fonts have many associated values. The right font (along with the right color and size) leaves a huge impact on readers and also shows your intention and emotion to put that font and make your identity. Font-family property can have values like family name (times, courier, arial, etc.) and generic-family (serif, sans-serif, cursive, fantasy, monospace)

In CSS, we use the font-family property to specify the family/ genre of the font on the webpage.

In CSS, there are five generic font families:

- **Serif** - All characters have a stroke attached at the endings.
- **Sans-serif** - These don't have small strokes attached at endings.
- **Cursive** - similar to human cursive writing, the letters are partially or fully connected.
- **Monospace** - all the letters have the same fixed width.
- **Fantasy** - Used for decoration and playful text.

```
font-family: family-name|generic-family|initial;
```

```

<!DOCTYPE html>
<html>
<head>
  <style>
    p.a {
      font-family: "Times New Roman",
Times, serif;
    }

    p.b {
      font-family: 'Roboto Mono',
monospace;
    }
  </style>
</head>
<body>
  <p class="a">This is a paragraph, shown in
the Times New Roman font.</p>
  <p class="b">This is a paragraph, shown in
the Roboto Mono font.</p>
</body>
</html>

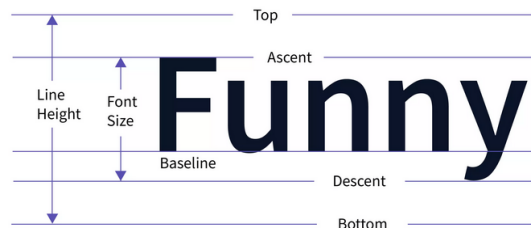
```

This is a paragraph, shown in the Times New Roman font.

This is a paragraph, shown in the Roboto Mono font.

▼ CSS Line-Height

Line height can be thought the distance between lines of text on a webpage. The line-height property in CSS allows us to define the height of the line box, i.e., the amount of space above and below the inline elements. It accepts the values such as normal, number, length, percentage, initial.



```
line-height: normal|number|length|percentage|initial;
```

```

<!DOCTYPE html>
<html>
<head>
  <style>
    div.a {
      line-height: normal;
    }

    div.b {
      line-height: 0.3;
    }

    div.c {
      line-height: 140%;
    }
  </style>
</head>
<body>
  <h2>line-height: normal (default):</h2>

  <div class="a">Lorem Ipsum is simply dummy
text of the printing and typesetting industry.
Lorem Ipsum has been the
  industry's standard dummy text ever since
the 1500s</div>

  <h2>line-height: 0.3 :</h2>

  <div class="b">Lorem Ipsum is simply dummy
text of the printing and typesetting industry.
Lorem Ipsum has been the
  industry's standard dummy text ever since
the 1500s, when an unknown printer took a galley
of type and scrambled

  </div>
  <h2>line-height: 140%:</h2>

  <div class="c">Lorem Ipsum is simply dummy
text of the printing and typesetting industry.
Lorem Ipsum has been the
  industry's standard dummy text ever since
the 1500s, when an unknown printer took a galley
of type</div>
</body>
</html>

```

line-height: normal (default):

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s

line-height: 0.3 :

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled

line-height: 140%:

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type

▼ CSS Font Stretch

The font-stretch property in CSS is used for stretching or compressing the text horizontally, and It is used to make the text wider or narrower. This property works only if there is a width-variance provided by the font family. It can have nine values from ultra-condensed to normal to ultra-expanded.

font-stretch: normal | semi-condensed | condensed | extra-condensed | ultra-condensed | semi-expanded | expanded | extra-

3

Web programming: Module III

▼ Introduction to JavaScript

▼ The Basics of JavaScript

▼ Variables

Variables are used to store data values in JavaScript. You can declare a variable using the `var`, `let` or `const` keyword.

1. `var`: Prior to the ES6 (ECMAScript 2015) standard, the `var` keyword was used to declare variables. Variables declared with `var` have function scope or global scope, depending on where they are declared. For example:

```
var name = "John";
```

2. `let`: Introduced in ES6, the `let` keyword allows you to declare block-scoped variables. Variables declared with `let` are limited to the block (enclosed by curly braces) in which they are defined. They can be reassigned later within the same block. For example:

```
let age = 25;
```

3. `const`: Also introduced in ES6, the `const` keyword is used to declare variables that have a constant value. Once a value is assigned to a `const` variable, it cannot be reassigned. `const` variables are block-scoped as well. For example:

```
const PI = 3.14159;
```

note that variables declared with `let` and `const` have block scope, whereas variables declared with `var` have function or global scope. Block scope refers to variables being accessible only within the block they are defined in (e.g., inside a loop or an if statement).

Example

```

var x = 5;      // Global scope variable
let y = 10;     // Block-scoped variable
const z = 15;   // Constant variable

if (x === 5) {
  let y = 20;   // y is a different variable in this block
  const z = 25; // z is a different variable in this block
  console.log(x, y, z); // Output: 5 20 25
}

console.log(x, y, z); // Output: 5 10 15

```

In this example, the `if` statement creates a new block scope, and the variables `y` and `z` defined within the block are separate from the outer variables with the same names.

▼ Data-types

▼ Primitive Data Types

- **Number:** Represents numeric values, including integers and floating-point numbers. For example: `let age = 25;`
- **String:** Represents sequences of characters enclosed in single or double quotes. For example: `let name = "John";`
- **Boolean:** Represents a logical value, either `true` or `false`. For example: `let isStudent = true;`
- **Null:** Represents the intentional absence of any object value. For example: `let myVariable = null;`
- **Undefined:** Represents an uninitialized variable or a variable without a value assigned to it. For example: `let myVariable;`
- **Symbol:** Represents a unique identifier. Symbols are often used as keys in objects to avoid naming conflicts. For example: `let id = Symbol("unique");`

▼ Complex Data Types

- **Object:** Represents a collection of key-value pairs or properties. Objects can be created using object literals or the `new Object()` constructor. For example:

```

let person = {
  name: "John",
  age: 25,
  isStudent: true
};

```

- **Array:** Represents an ordered list of values, enclosed in square brackets. Arrays can contain elements of any data type. For example:

```

let numbers = [1, 2, 3, 4, 5];

```

▼ Operators

- **Arithmetic operators** are used to perform mathematical operations on numbers. For example, the addition operator (+) adds two numbers together, the subtraction operator (-) subtracts two numbers, the multiplication operator (*) multiplies two numbers together, and the division operator (/) divides two numbers.
- **Assignment operators** are used to assign values to variables. For example, the assignment operator (=) assigns the value of the expression on the right side of the operator to the variable on the left side.
- **Comparison operators** are used to compare two values and determine if they are equal, greater than, less than, or not equal. For example, the equal to operator (==) compares two values and returns true if they are equal, and false if they are not.
- **Logical operators** are used to combine logical expressions. For example, the logical AND operator (&&) returns true if both of the expressions it is applied to are true, and false if either of them is false.
- **Bitwise operators** are used to perform bitwise operations on numbers. For example, the bitwise AND operator (&) performs a Boolean AND operation on each bit of its integer arguments.

Operator	Name	Symbol
Arithmetic operators	Addition	+
	Subtraction	-
	Multiplication	*
	Division	/
	Modulus	%
Assignment operators	Assignment	=
	Addition assignment	+=
	Subtraction assignment	-=
	Multiplication assignment	*=
	Division assignment	/=
Comparison operators	Modulus assignment	%=
	Equal to	==
	Not equal to	!=
	Greater than	>
	Less than	<
Logical operators	Greater than or equal to	>=
	Less than or equal to	<=
	Logical AND	&&

Operator	Name	Symbol
	Logical OR	
	Logical NOT	!
Bitwise operators	Bitwise AND	&
	Bitwise OR	
	Bitwise XOR	^
	Bitwise NOT	~
	Conditional	? :
Ternary operator	Conditional	? :

•

▼ Control Structure

▼ Conditional statements

▼ if Statement

The if statement is used to execute a block of code if a certain condition is true. It has the following syntax:

```
if (condition) {
  // Code to be executed if the condition is true
} else {
  // Code to be executed if the condition is false
}
```

Example:

```
let num = 10;

if (num > 0) {
  console.log("The number is positive.");
} else {
  console.log("The number is non-positive.");
}
//Output: The number is positive.
```

▼ Ternary Statement

The ternary operator provides a more concise way to write conditional statements. It is a shorthand form of an if-else statement. The syntax of the ternary operator is as follows:

```
condition ? expression1 : expression2
```

Example:

```
let num = 5;
let result = num > 0 ? "Positive" : "Negative or zero";
```

```
console.log(result);  
//Output: Positive
```

▼ Loop statements

▼ switch Statement

The switch statement allows you to select one of many code blocks to be executed based on the value of an expression. It has the following syntax:

```
switch (expression) {  
  case value1:  
    // Code to be executed if the expression matches value1  
    break;  
  case value2:  
    // Code to be executed if the expression matches value2  
    break;  
  // More cases...  
  default:  
    // Code to be executed if none of the cases match  
    break;  
}
```

Example:

```
let day = "Monday";  
  
switch (day) {  
  case "Monday":  
    console.log("It's the beginning of the week.");  
    break;  
  case "Friday":  
    console.log("It's the end of the week.");  
    break;  
  default:  
    console.log("It's another day of the week.");  
    break;  
}  
//Output: It's the beginning of the week.
```

▼ for Loop

The for loop is used to repeat a block of code for a specified number of times. It has the following syntax:

```
for (initialization; condition; iteration) {  
  // Code to be executed in each iteration  
}
```

Example:

```
for (let i = 1; i <= 5; i++) {  
  console.log(i);  
}
```

```
//Output:  
//1  
//2  
//3  
//4  
//5
```

▼ while Loop

The while loop is used to repeat a block of code as long as a condition is true. It has the following syntax:

```
while (condition) {  
    // Code to be executed in each iteration  
}
```

Example:

```
let count = 0;  
  
while (count < 5) {  
    console.log("Count: " + count);  
    count++;  
}  
  
//Output:  
//Count: 0  
//Count: 1  
//Count: 2  
//Count: 3  
//Count: 4
```

▼ Do-while Loop

The do-while loop is similar to the while loop, but it always executes the block of code at least once before checking the condition. It has the following syntax:

```
do {  
    // Code to be executed in each iteration  
} while (condition);
```

Example:

```
let i = 0;  
  
do {  
    console.log(i);  
    i++;  
} while (i < 5);  
  
//Output:  
//0  
//1
```

```
//2
//3
//4
```

▼ Functions

Functions are reusable blocks of code that perform a specific task or calculate a value. They play a crucial role in structuring and organizing code, promoting reusability and modularity. Here's an overview of functions in JavaScript, including function definition, types, function calls, and return values.

▼ Function Definition

A function in JavaScript is defined using the `function` keyword, followed by the function name, a pair of parentheses `()`, and a block of code enclosed in curly braces `{ }`. The basic syntax for defining a function is as follows:

```
function functionName(parameter1, parameter2, ...) {
  // Code to be executed
}
```

Example:

```
function addNumbers(num1, num2) {
  let sum = num1 + num2;
  console.log("The sum is: " + sum);
}
```

▼ Function Calls

To execute a function, you need to call it by its name, followed by a pair of parentheses `()`. If the function has parameters, you can pass the values inside the parentheses.

Example

```
function greet(name) {
  console.log("Hello, " + name + "!");
}

greet("Alice"); // Output: Hello, Alice!
```

▼ return values

Functions can return values using the `return` statement. The `return` statement stops the execution of the function and sends a value back to the caller.

```
function multiply(num1, num2) {
  return num1 * num2;
}
```

```
console.log("The result is: " + multiply(3,4)); // Output: The result is: 12
```

▼ Function Types

In JavaScript, there are two main types of functions: named functions and anonymous functions (also known as function expressions).

▼ Named Functions

Named functions are defined with a specific name and can be called by their name. They are commonly used when you want to reuse the same code block in multiple places within your program.

Example

```
function greet(name) {  
    console.log("Hello, " + name + "!");  
}  
  
// Calling the named function  
greet("John");
```

▼ Anonymous Functions

Anonymous functions, as the name suggests, do not have a specified name. They are often assigned to variables or used as callback functions.

Example

```
let multiply = function(num1, num2) {  
    return num1 * num2;  
};  
  
// Calling the anonymous function  
console.log("The result is: " + multiply(3, 4));
```

▼ Pop up Boxes

we can use pop-up boxes to interact with the user by displaying messages or prompts. There are three main types of pop-up boxes: alert, confirm, and prompt. These boxes provide a way to communicate with the user and gather input if needed.

▼ Alert Boxes

The alert box displays a message to the user and only requires an acknowledgement from them. It has a simple OK button to dismiss the box. Here's the syntax to use an alert box:

```
alert("This is an alert message.");
```

When this line of code is executed, an alert box will pop up on the screen with the specified message.

▼ Confirm Boxes

The confirm box allows the user to confirm or cancel an action. It displays a message along with two buttons: OK and Cancel. The confirm box returns a Boolean value (`true` if OK is clicked, `false` if Cancel is clicked). Here's an example:

```
let result = confirm("Are you sure you want to delete this item?");
if (result) {
  // Code to delete the item
} else {
  // Code to cancel the deletion
}
```

The user's choice is stored in the `result` variable, which can be used to perform different actions based on the selection.

▼ Prompt Boxes

The prompt box allows the user to input data by providing a text input field along with a message. It returns the value entered by the user as a string. Here's an example:

```
let name = prompt("Please enter your name:");
console.log("Hello, " + name + "!");
```

In this example, the user will see a prompt box asking them to enter their name. The value entered by the user will be stored in the `name` variable, which can be used for further processing.

▼ Advanced JavaScript

▼ JavaScript Arrays

An array is a special kind of object that can store a collection of data. Arrays are zero-indexed, which means that the first element of an array is at index 0, the second element is at index 1, and so on.

Arrays can be declared in a few different ways:

- Using square brackets:

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
```

- Using the `new Array()` constructor:

```
const fruits = new Array("Banana", "Orange", "Apple", "Mango");
```

- Using the `Array.from()` method:

```
const fruits = Array.from(["Banana", "Orange", "Apple", "Mango"]);
```

Once an array is declared, you can access its elements using their indexes. For example, to access the first element of the `fruits` array, you would use the following code:

```
const firstFruit = fruits[0];
```

Array Looping

You can also loop through an array using a for loop:

```
for (let i = 0; i < fruits.length; i++) {  
  console.log(fruits[i]);  
}
```

▼ **Objects**

Objects are one of the fundamental data types and are used to store collections of key-value pairs. They are versatile and can represent various entities, such as real-world objects, data structures, and more.

Creating Objects:

There are a few ways to create objects in JavaScript:

1. Object Literal:

```
const person = {  
  name: 'John',  
  age: 25,  
  profession: 'Engineer'  
};
```

2. Using the `new` keyword with the Object constructor:

```
const person = new Object();  
person.name = 'John';  
person.age = 25;  
person.profession = 'Engineer';
```

3. Using constructor functions

```
function Person(name, age, profession) {  
  this.name = name;  
  this.age = age;
```

```
    this.profession = profession;
  }

  const person = new Person('John', 25, 'Engineer');
```

Accessing Object Properties:

You can access object properties using dot notation or bracket notation:

```
console.log(person.name); // Output: John
console.log(person['age']); // Output: 25
```

Modifying Object Properties:

You can modify object properties by assigning new values to them:

```
person.age = 30;
person['profession'] = 'Developer';
```

Adding and Deleting Properties:

You can add new properties to an existing object or delete existing properties:

```
person.location = 'New York';
delete person.profession;
```

Object Methods:

Objects can also have methods, which are functions attached to the object. These methods can perform operations on the object's properties or provide additional functionality:

```
const person = {
  name: 'John',
  age: 25,
  greet: function() {
    console.log('Hello, my name is ' + this.name);
  }
};

person.greet(); // Output: Hello, my name is John
```

▼ **Number Object**

The `Number` object is a built-in object that provides functionality and methods for working with numbers. It is also used to create instances of numbers as objects.

When you create a `Number` object using the `new Number(value)` syntax, you are creating a new instance of the `Number` object. The `value` parameter represents the numeric value that you want to assign to the `Number` object.

```
var n = new Number(10);
console.log(typeof n); // Output: "object"
console.log(n.valueOf()); // Output: 10
```

The `Number` object has various properties and methods that allow you to perform operations on numbers. Some commonly used properties include:

- **`Number.MAX_VALUE`**: The maximum representable numeric value in JavaScript.
- **`Number.MIN_VALUE`**: The minimum representable positive numeric value in JavaScript.
- **`Number.NaN`**: A special value representing "Not-a-Number" when a numeric value is not a valid number.
- **`Number.POSITIVE_INFINITY`**: A special value representing positive infinity.
- **`Number.NEGATIVE_INFINITY`**: A special value representing negative infinity.

Example

```
console.log(Number.MAX_VALUE); // Output: 1.7976931348623157e+308
console.log(Number.MIN_VALUE); // Output: 5e-324
console.log(Number.NaN); // Output: NaN
console.log(Number.POSITIVE_INFINITY); // Output: Infinity
console.log(Number.NEGATIVE_INFINITY); // Output: -Infinity
```

In addition to properties, the `Number` object also provides methods for performing operations on numbers. Some commonly used methods include `toFixed()`, `toPrecision()`, and `toString()`. These methods allow you to format numbers and convert them to strings in different ways.

```
var num = new Number(10.456);
console.log(num.toFixed(2)); // Output: "10.46"
console.log(num.toPrecision(3)); // Output: "10.5"
console.log(num.toString()); // Output: "10.456"
```

▼ String Object

A string object in JavaScript is a sequence of characters that can be manipulated using a variety of methods and properties. Strings are immutable, which means that they cannot be changed once they are created.

Creating a string is as simple as assigning a value to a variable:

```
let message = "Hello, World!";
```

You can also use the `String()` constructor to create a string object:

```
let strObject = new String("Hello, World!");
```

The String object provides various methods for manipulating and working with strings. Here are a few commonly used methods:

1. `length` : Returns the length of the string.

```
let message = "Hello, World!";  
console.log(message.length); // Output: 13
```

2. `toUpperCase()` : Converts the string to uppercase.

```
toUpperCase(): Converts the string to uppercase.
```

3. `toLowerCase()` : Converts the string to lowercase.

```
let message = "Hello, World!";  
console.log(message.toLowerCase()); // Output: hello, world!
```

4. `charAt(index)` : Returns the character at the specified index.

```
let message = "Hello, World!";  
console.log(message.charAt(0)); // Output: H  
console.log(message.charAt(4)); // Output: o
```

▼ Date Object

Date object is used to work with dates and times. It provides methods for creating, manipulating, and formatting dates.

Creating a Date object is done using the `new Date()` constructor. There are several ways to create a Date object:

1. **Without any arguments:** This creates a Date object representing the current date and time.

```
let currentDate = new Date();
```

2. **With a specific date and time:** You can pass in the year, month (0-11), day, hour, minute, second, and millisecond as arguments to specify a particular date and time.

```
let specificDate = new Date(2023, 6, 10, 12, 30, 0, 0); // July 10, 2023, 12:30:00
```

The Date object provides various methods to access and manipulate different components of the date and time. Here are a few commonly used methods:

1. `getFullYear()` : Returns the four-digit year of the date.

```
let year = specificDate.getFullYear();
```

2. `getMonth()` : Returns the month (0-11) of the date.

```
let month = specificDate.getMonth();
```

3. `getDate()` : Returns the day of the month (1-31) of the date.

```
let day = specificDate.getDate();
```

4. `getDay()` : Returns the day of the week (0-6, where 0 is Sunday) of the date.

```
let weekDay = specificDate.getDay();
```

5. `getHours()` , `getMinutes()` , `getSeconds()` : Return the hour, minute, and second of the date, respectively.

```
let hours = specificDate.getHours();  
let minutes = specificDate.getMinutes();  
let seconds = specificDate.getSeconds();
```

▼ Math Object

The `Math` object is a built-in global object that provides mathematical constants and functions for performing various mathematical operations. It does not need to be instantiated and can be used directly with its properties and methods.

common properties and methods of the `Math` object:

1. Mathematical Constants:

- `Math.PI` : Represents the mathematical constant π (pi), which is the ratio of a circle's circumference to its diameter.
- `Math.E` : Represents Euler's number, the base of the natural logarithm.

2. Trigonometric Functions:

- `Math.sin(x)` : Returns the sine of the angle `x` (in radians).
- `Math.cos(x)` : Returns the cosine of the angle `x` (in radians).
- `Math.tan(x)` : Returns the tangent of the angle `x` (in radians).
- `Math.atan(x)` : Returns the arctangent (inverse tangent) of `x` (in radians).

3. Exponential and Logarithmic Functions:

- `Math.exp(x)` : Returns the value of Euler's number raised to the power of `x`.
- `Math.log(x)` : Returns the natural logarithm (base e) of `x`.
- `Math.log10(x)` : Returns the base-10 logarithm of `x`.

4. Rounding and Random Functions:

- `Math.round(x)` : Rounds a number to the nearest integer.
- `Math.floor(x)` : Rounds a number down to the nearest integer.
- `Math.ceil(x)` : Rounds a number up to the nearest integer.
- `Math.random()` : Generates a pseudo-random number between 0 (inclusive) and 1 (exclusive).

5. Additional Functions:

- `Math.abs(x)` : Returns the absolute value of `x`.
- `Math.sqrt(x)` : Returns the square root of `x`.
- `Math.pow(x, y)` : Returns `x` raised to the power of `y`.
- `Math.min(x1, x2, ..., xn)` : Returns the minimum value among the provided arguments.
- `Math.max(x1, x2, ..., xn)` : Returns the maximum value among the provided arguments.

Example

```
const radius = 5;
const circumference = 2 * Math.PI * radius;
console.log(`Circumference: ${circumference}`);

const angle = Math.PI / 4;
const sineValue = Math.sin(angle);
console.log(`Sine of ${angle} radians: ${sineValue}`);

const randomNumber = Math.random(); // Generates a random number between 0 (inclusive) and 1 (exclusive)
console.log(`Random number: ${randomNumber}`);
```

In this example, we use the `Math.PI` property to calculate the circumference of a circle, the `Math.sin()` method to find the sine of an angle, and the `Math.random()` method to generate a random number.

▼ Event Handling

Event handling allows you to respond to user interactions and other events that occur in the web browser. You can use event handlers to trigger specific functions or actions when events like mouse clicks, keyboard input, form submissions, or other interactions take place.

Events are :

1. Event Listener:

An event listener is a function that waits for a specific event to occur. It "listens" to the event and triggers the associated function when the event occurs. You can attach event listeners to HTML elements to respond to events.

2. Event Object:

When an event occurs, the browser generates an event object that contains information about the event. This object can be accessed in the event handler function to get details about the event (e.g., mouse coordinates, key pressed, etc.).

3. Event Types:

JavaScript supports various types of events. Some common event types include:

- Mouse Events: `click`, `dblclick`, `mouseover`, `mouseout`, etc.
- Keyboard Events: `keydown`, `keyup`, `keypress`.
- Form Events: `submit`, `change`, `input`, etc.
- Focus Events: `focus`, `blur`.
- Window Events: `load`, `resize`, `scroll`, etc.

Example

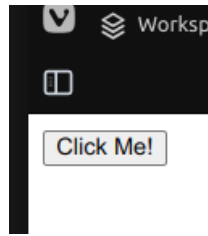
Event handling for a mouse click event:

```
<!DOCTYPE html>
<html>
<head>
  <title>Event Handling in JavaScript</title>
</head>
<body>
  <button id="myButton">Click Me!</button>

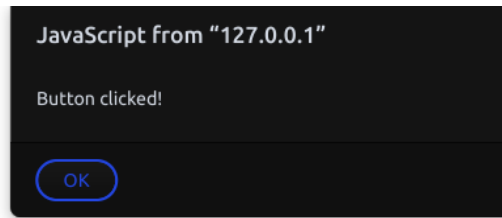
  <script>
    // Function to handle the click event
    function handleClick(event) {

      // Your desired action here
      alert('Button clicked!');
    }

    // Adding an event listener to the button element
    const button = document.getElementById('myButton');
    button.addEventListener('click', handleClick);
  </script>
</body>
</html>
```



When Clicked>>>



▼ JavaScript RegExp

<https://youtu.be/sXQxhojSdZM>

Regex (regular expressions) in JavaScript are used for pattern matching and manipulating strings. JavaScript provides a built-in `RegExp` object to work with regular expressions. Regular expressions are powerful tools that allow you to search, replace, and extract specific parts of a string based on a pattern.

Basic overview of using regular expressions in JavaScript:

1. Creating a regular expression:

You can create a regular expression using the `RegExp` constructor or by using a regex literal enclosed in forward slashes.

Using `RegExp` constructor:

```
const regex = new RegExp('pattern');
```

or

Using regex literal:

```
const regex = /pattern/;
```

2. Testing for matches:

You can use the `test()` method of the `RegExp` object to check if a pattern matches a string.

```
const regex = /hello/;
const str = 'Hello, world!';

console.log(regex.test(str)); // Output: false
```

3. Matching:

To extract matches from a string, you can use the `match()` method of the string object.

```
const regex = /world/;
const str = 'Hello, world!';

const result = str.match(regex);
console.log(result); // Output: ["world"]
```

4. Replacing :

You can use the `replace()` method of the string object to replace matches with a specified replacement string.

```
const regex = /apple/g;
//The 'g' is a flag which helps to select all the matching words in the sentence
const str = ' apple apple apple ';
const result = str.replace(regex, 'orange');
console.log(result); // Output: "orange orange orange."
```

5. Matching Groups :

Parentheses `()` can be used to create matching groups that allow you to extract specific parts of the matched string.

```
const regex = /(\d{2})-(\d{2})-(\d{4})/;
const str = 'Today is 27-07-2023, and it is a sunny day.';

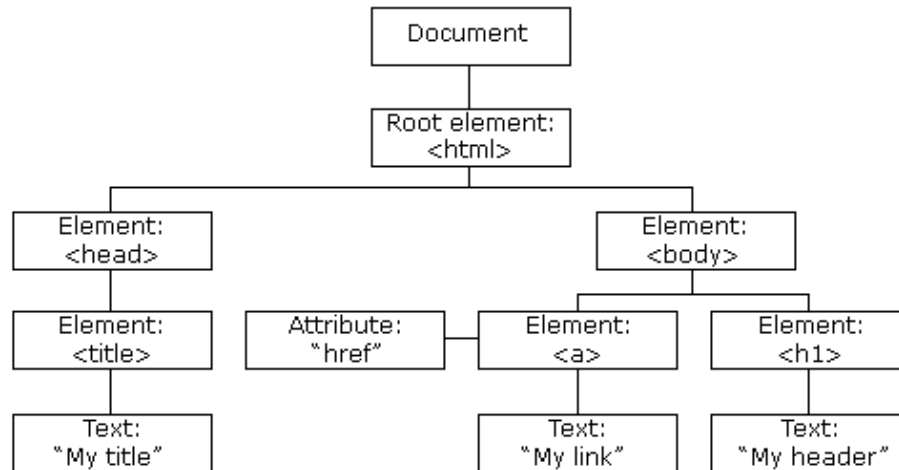
const result = str.match(regex);
console.log(result); // Output: ["27-07-2023", "27", "07", "2023"]
```

Flags:

You can use flags with the regex literal to modify the behavior of the regular expression.

- **g** (global): Matches all occurrences in the string, not just the first one.
- **i** (ignore case): Performs a case-insensitive match.
- **m** (multiline): Allows `^` and `$` to match the start/end of lines.

▼ JavaScript DOM



The DOM is a programming interface provided by browsers that allows developers to interact with and manipulate the content and structure of HTML documents. When a web page is loaded in the browser, the browser creates a tree-like representation of the page's structure, where each HTML element becomes a node in the tree, and attributes and text content become properties of those nodes. The DOM provides methods and properties that allow developers to access, modify, and manipulate these nodes.

For example, using the DOM, you can:

- Access and modify HTML elements' attributes and content.
- Add or remove elements from the page dynamically.
- Attach event listeners to elements to respond to user interactions.
- Change CSS styles of elements.

However, directly manipulating the DOM can be costly in terms of performance, especially when dealing with complex web applications with frequent updates, as changes to the DOM can trigger expensive reflows and repaints. This is where the Virtual DOM comes into play.

▼ JavaScript validation

JavaScript validation is a technique used to validate user input on the client-side before it is submitted to the server. This ensures that the data entered by the user meets specific criteria or follows a particular format. Client-side validation helps to provide a more responsive and user-friendly experience by catching errors early and reducing unnecessary server requests.

Example

```
<!DOCTYPE html>

<html>

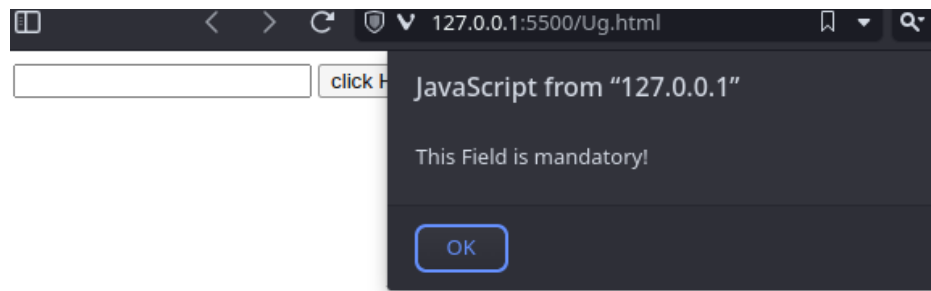
<body>
```

```
<input type="text" id="one">
<button onclick="samplefun()">click Here!</button>

<script>
function samplefun(){
    var a = document.getElementById('one').value
    if(a==""){
        alert('This Field is mandatory!')
    }
}

</script>
</body>
</html>
```

Output



In this example, we have a simple HTML form with a text input field and a button. The goal of the JavaScript function `samplefun()` is to validate the input field to ensure it is not empty when the button is clicked.

4

Web programming : module IV

▼ Introduction to XML

▼ The syntax of XML

XML (eXtensible Markup Language) is a markup language that uses tags to describe data. XML is a text-based format, which means that it can be read and understood by humans as well as machines.

The syntax of XML is very strict. The basic rules of XML syntax are:

- ▼ All XML documents must have a root element. The root element is the outermost element in the document.

```
<root>
  <child>
    <subchild>....</subchild>
  </child>
</root>
```

Eg:

```
<?xml version="1.0" encoding="UTF-8"?>
<person>
  <name>John Doe</name>
  <age>30</age>
  <address>123 Main Street</address>
</person>
```

The `person` element is the root element of the document. It encloses the `name`, `age`, and `address` elements. The `name`, `age`, and `address` elements are child elements of the `person` element.

The root element is also used to identify the type of data in the document. In the example above, the root element is called `person`, which indicates that the document contains data about a person.

- ▼ All XML elements must have a start tag and an end tag. The start tag identifies the element, and the end tag closes the element.

```
<p>This is a paragraph.</p>
<br />
```

The `<p>` tag is the start tag for a paragraph element in XML. The `</p>` tag is the end tag for a paragraph element in XML. The paragraph element is used to contain a block of text.

The `<br />` tag is a line break tag in XML. It is used to insert a line break in the text. The line break tag does not have a closing tag.

▼ **The names of XML elements are case-sensitive.**

The `<p>` tag and the `</p>` tag are case-sensitive, so you must use the correct capitalization. The `<br />` tag is not case-sensitive.

▼ **XML attributes are name-value pairs that are associated with an element. Attributes are enclosed in the start tag of the element, and they are separated by spaces.**

```
<person name="John Doe">
...
</person>
```

The `name` attribute is associated with the `person` element, and the value of the `name` attribute is `John Doe`.

▼ **XML comments are used to provide information about the XML document. Comments are enclosed in `<!--` and `-->`.**

```
<!-- The `name` element is the name of the person. -->
<name>John Doe</name>
```

▼ Document Structure

1. **XML Declaration:** An optional statement at the beginning of the document that specifies the XML version being used and the character encoding. It looks like this:

```
<?xml version="1.0" encoding="UTF-8"?>
```

2. **Root Element:** The top-level element that encloses all other elements in the XML document. It serves as the entry point for parsing the document and must be unique. It should be wrapped around all other elements. For example:

```
<root>
  <!-- Other elements go here -->
</root>
```

3. **Elements:** Elements are the building blocks of an XML document. They are defined by a start tag, an end tag, and the content they enclose. Elements can be nested inside other elements to create a hierarchical structure. For example:

```
<element_name>Content goes here</element_name>
```

4. **Attributes:** Elements can have attributes, which provide additional information about the element. Attributes are defined within the start tag of an element and take the form of name-value pairs. For example:

```
<person name="John" age="30" />
```

5. **Comments:** Comments are used to add explanatory notes within the XML code and are ignored by the XML parser during processing. Comments start with `<!--` and end with `-->`, like this:

```
<!-- This is a comment -->
```

Example

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="fiction">
    <title lang="en">Harry Potter and the Sorcerer's Stone</title>
    <author>J.K. Rowling</author>
    <year>1997</year>
  </book>
  <book category="non-fiction">
    <title lang="en">Sapiens: A Brief History of Humankind</title>
    <author>Yuval Noah Harari</author>
    <year>2011</year>
  </book>
</bookstore>
```

In this example, the root element is `<bookstore>`, which contains two `<book>` elements, each having attributes (category). The `<book>` elements, in turn, contain child elements like `<title>`, `<author>`, and `<year>`, as well as attributes (lang) within the `<title>` element.

▼ Namespaces

Namespaces in XML are used to avoid naming conflicts between elements and attributes when combining XML documents from different sources. They provide a way to uniquely identify elements and attributes by associating them with a namespace URI (Uniform Resource Identifier). This allows XML documents to use the same element or attribute names within different namespaces without ambiguity.

A namespace declaration is typically done at the root element of an XML document using the `xmlns` attribute. The syntax for declaring a namespace is as follows:

The syntax for declaring a namespace is:

```
xmlns:prefix="URI"
```

- `prefix` is a short identifier that is used to refer to the namespace in the document.
- `URI` is the Uniform Resource Identifier (URI) that identifies the namespace.

Example

```
<?xml version="1.0" encoding="UTF-8"?>
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <title>My Document</title>
  </head>
  <body>
    <p>This is my document.</p>
  </body>
</html>
```

Once a namespace has been declared, it can be used to refer to elements and attributes from that namespace. For example, the above XML document uses the `http://www.w3.org/1999/xhtml` namespace to refer to the `html` element

▼ XML Schemas

https://youtu.be/56pR_5rO-m4

<https://youtu.be/1BjmZHRHDv0>

XML Schema, often referred to as XSD (XML Schema Definition), is a specification used to describe the structure, content, and data types of XML documents. It provides a way to define rules and constraints for the elements and attributes within an XML document, ensuring that the XML data adheres to a specific structure.

XML Schema is commonly used for:

1. **Data Validation:** XML Schema allows you to define the allowed structure and content of XML documents. By validating XML against a schema, you can ensure that the data is well-formed and conforms to the specified rules.
2. **Data Modeling:** XML Schema provides a way to create a model for the XML data. You can define elements, attributes, their data types, relationships, and cardinalities, thereby establishing a blueprint for the XML document.
3. **Interoperability:** XML Schema promotes interoperability between different systems by providing a standardized way to describe XML data structures. This allows different applications to exchange data seamlessly when they follow the same XML Schema.

XML Schema itself is written in XML format and uses elements and attributes to define the structure of an XML document. A schema consists of several components, including:

- **Elements:** Describes the structure and content of XML elements, including their names, data types, occurrence constraints, and nested elements.

- **Attributes:** Defines attributes that can be associated with XML elements, along with their data types and constraints.
- **Data Types:** Specifies the allowable data types for element content and attribute values, such as strings, numbers, dates, etc.
- **Complex Types:** Describes elements that contain other elements or attributes. Complex types can have sequences, choices, and other compositor elements to define the order and occurrence of child elements.
- **Simple Types:** Represents atomic data types that are not composed of other elements or attributes.

▼ Displaying Raw XML Documents

1. Using a Text Editor:

- Open your preferred text editor (e.g., Notepad, Notepad++, Sublime Text, Visual Studio Code, etc.).
- Copy and paste the XML content into the text editor.
- Save the file with a `.xml` extension, for example, `myfile.xml`.
- You can now view and edit the raw XML content in the text editor.

2. Using an XML Viewer or Browser Extension:

- There are various online XML viewers available that allow you to paste or upload the XML content to display it in a structured format.
- Alternatively, you can use browser extensions or add-ons that allow you to view XML documents directly in your web browser with syntax highlighting.

3. Using Code Editor with XML Syntax Highlighting:

- If you prefer a code editor with advanced features, you can use editors like Visual Studio Code, Sublime Text, or Atom. These editors often have XML syntax highlighting, making it easier to read and navigate XML files.

4. Viewing XML in the Browser:

- You can also open an XML document in a web browser. Browsers like Google Chrome and Firefox can display raw XML documents with syntax highlighting.
- Simply save the XML content in a file with `.xml` extension, then open it with your web browser. The browser will render the XML with syntax highlighting for better readability.

▼ Displaying XML Documents with CSS

Consider the below XML Code :

```
<?xml version="1.0"?>
<bookstore>
  <book>
```

```

    <title>Book 1</title>
    <author>Author 1</author>
    <genre>Fiction</genre>
  </book>
  <book>
    <title>Book 2</title>
    <author>Author 2</author>
    <genre>Non-Fiction</genre>
  </book>
</bookstore>

```

You can create an HTML file that includes the XML content and link to an external CSS file for styling:

```

<!DOCTYPE html>
<html>
<head>
  <title>Styled XML Document</title>
  <link rel="stylesheet" type="text/css" href="styles.css">

  <!-- Here is the XML Part mentioned above -->

  <pre>
    <?xml version="1.0"?>
    <bookstore>
      <book>
        <title>Book 1</title>
        <author>Author 1</author>
        <genre>Fiction</genre>
      </book>
      <book>
        <title>Book 2</title>
        <author>Author 2</author>
        <genre>Non-Fiction</genre>
      </book>
    </bookstore>
  </pre>

  <!-- Till HERE-->

</body>
</html>

```

Let the `styles.css` be:

```

bookstore {
  display: block;
  font-weight: bold;
}

book {
  display: block;
  margin-bottom: 10px;
}

title {
  color: blue;
}

author {
  font-style: italic;
}

```

```

}

genre {
  text-transform: uppercase;
}

```

The above CSS styles the `<bookstore>`, `<book>`, `<title>`, `<author>`, and `<genre>` elements. It's important to note that you can only style elements that are explicitly defined in the XML structure.

Output of the HTML File is 

Author 1
FICTION

Author 2
NON-FICTION

▼ XSLT style sheets

XSLT (Extensible Stylesheet Language Transformations) is a language used for transforming XML documents into other formats, such as HTML, plain text, or other XML structures. XSLT uses XPath to navigate and manipulate the XML data.

Example

Consider the `XML` file below:

`movie.xml`

```

<?xml-stylesheet href="movie.xsl" type="text/xsl"?>

<collection>
<movie>
  <title> Openheimer</title>
  <year>2023</year>
  <genre>Biopic</genre>
</movie>

<movie>
  <title> barbie</title>
  <year>2023</year>
  <genre>Fun</genre>
</movie>

<movie>
  <title> 1983</title>
  <year>2018</year>
  <genre>Drama</genre>
</movie>

</collection>

```

Suppose we need it to be arranged in a separate template and import it as a `HTML` file then:

`movies.xsl`

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl='http://www.w3.org/1999/XSL/Transform'>

<xsl:template match="/collection">
<html>
  <body>
    <xsl:for-each select="movie">
      <xsl:value-of select="title"/><br/>
      <xsl:value-of select="year"/> <br/>
      <xsl:value-of select="genre"/><br/>
    </xsl:for-each>
  </body>
</html>

</xsl:template>
</xsl:stylesheet>
```

These files should be contained in a single folder .

The code

```
xsltproc -o output.html movies.xsl movie.xml
```

convert the `.xml` to `.html` using the template mentioned in `.xsl`

output

`output.html`

```
<html><body> Openheimer<br>2023<br>Biopic<br><br> barbie<br>2023<br>Fun<br><br> 1983<br>2018<br>Biopic<br><br>
</body></html>
```

output in browser

Openheimer
2023
Biopic

barbie
2023
Fun

1983
2018
Biopic

▼ XML Applications

1. **Web Services and APIs:** XML is often used as a data interchange format in web services and APIs (Application Programming Interfaces). It allows different systems and platforms to communicate and exchange data in a standardized manner.
2. **Configuration Files:** Many software applications use XML for storing configuration settings. XML's hierarchical structure is ideal for representing complex configuration data.
3. **Data Interchange:** XML is commonly used for exchanging data between different applications or systems. It enables seamless data sharing and interoperability.
4. **RSS and Atom Feeds:** XML-based formats like RSS (Really Simple Syndication) and Atom are used for publishing and subscribing to content updates from websites, blogs, and news sources.
5. **Document Formats:** XML is utilized in various document formats like DocBook and DITA (Darwin Information Typing Architecture) for creating structured and reusable content.
6. **Database Integration:** XML can be used to facilitate data exchange between databases and applications, especially in heterogeneous environments.
7. **Middleware Messaging:** XML is used in middleware technologies to facilitate communication between distributed systems, providing a standardized message format for communication between different components.
8. **Office Applications:** Some office productivity tools, such as Microsoft Office (Word, Excel, PowerPoint), utilize XML-based formats for document storage and manipulation.
9. **Data Representation:** XML serves as a data representation format for various applications, including scientific data, financial data, and data exchange in supply chain management.
10. **Configuration Management:** XML is employed in configuration management tools to manage software and hardware configurations.
11. **Mobile Applications:** XML is used in mobile application development to store and exchange data between the mobile app and backend services.
12. **Medical Data:** XML is used in healthcare and medical applications to represent patient records, medical codes, and other clinical data.

▼ DHTML

DHTML stands for "Dynamic HTML," which is a combination of technologies used to create dynamic and interactive web pages. It was popular in the late 1990s and early 2000s and marked an important milestone in web development.

“DHTML is not a specific technology or programming language; instead, it is a collection of technologies that work together to enhance the functionality and interactivity of web pages.”

▼ Components of DHTML

1. **HTML (Hypertext Markup Language):** The foundation of web pages, defining the structure and content of the page.
2. **CSS (Cascading Style Sheets):** Used to control the presentation and layout of HTML elements, allowing for better control of the visual appearance.
3. **JavaScript:** The programming language used to add interactivity to web pages. JavaScript is the primary driver of dynamic changes in DHTML, allowing developers to manipulate HTML and CSS in real-time, respond to user actions, and create animations and effects.
4. **DOM (Document Object Model):** A programming interface for HTML and XML documents. It represents the structure of a web page as a hierarchical tree of objects, which can be accessed and manipulated using JavaScript. DOM enables dynamic modification of the content and structure of the page.
5. **Event Handling:** DHTML allows developers to capture and respond to various user actions or events, such as mouse clicks, keyboard inputs, and form submissions. JavaScript is used to handle these events and trigger appropriate actions.

▼ Difference between HTML and DHTML

Feature	HTML (HyperText Markup Language)	DHTML (Dynamic HyperText Markup Language)
Definition	Markup language used for creating the structure and content of web pages.	Extension of HTML that combines JavaScript, CSS, and HTML for dynamic and interactive content.
Interactivity	Static content; no built-in support for dynamic or interactive elements.	Enables dynamic and interactive elements, allowing content to change, move, or update without page reload.
Animation	No native support for animations.	Supports animations through JavaScript and CSS transitions.
Events	Limited support for handling events; primarily links and form submissions.	Enables event handling for elements, such as click, hover, etc., with JavaScript.
User Experience	Generally, provides a basic and straightforward user experience.	Enhances user experience with interactive elements and real-time updates.
Development	Simple to learn and implement for static web pages.	Requires proficiency in JavaScript, CSS, and HTML for implementation.

Feature	HTML (HyperText Markup Language)	DHTML (Dynamic HyperText Markup Language)
Examples	Creating headings, paragraphs, images, links, etc.	Building sliders, dropdown menus, dynamic forms, and interactive UI components.

▼ DHTML JavaScript

It refers to the combination of Dynamic HyperText Markup Language (DHTML) and JavaScript (Js) to create interactive and dynamic web content. DHTML is not a separate language; rather, it's a term used to describe the use of JavaScript, CSS, and HTML together to achieve dynamic effects and interactivity on web pages.

JavaScript is a powerful scripting language that can be used to manipulate the Document Object Model (DOM) of an HTML document. The DOM represents the structure of a web page, and by using JavaScript, developers can interact with and modify the content and behavior of the page in real-time

You can include JavaScript code directly within the HTML file using the `<script>` element inside the `<head>` or `<body>` section.

Inline scripting

```
<!DOCTYPE html>
<html>
<head>
  <title>Inline Scripting Example</title>
  <!-- JavaScript code inside <script> tags -->
  <script>
    // Your JavaScript code goes here
    // For example:
    function greet() {
      alert('Hello, world!');
    }
  </script>
</head>
<body>
  <h1>Inline Scripting Example</h1>
  <!-- You can call the JavaScript function defined above -->
  <button onclick="greet()">Click Me</button>
</body>
</html>
```

External Scripting

save as **script.js**

```
function greet() {
  alert('Hello, world!');
}
```

web.html

```
<!DOCTYPE html>
<html>
```

```

<head>
  <title>External Script Example</title>
  <!-- Linking the external JavaScript file -->
  <script src="script.js"></script>
</head>
<body>
  <h1>External Script Example</h1>
  <!-- You can call the JavaScript function defined in the external file -->
  <button onclick="greet()">Click Me</button>
</body>
</html>

```

here we link the external JavaScript file to your HTML file using the `<script>` element

▼ DHTML CSS

DHTML (Dynamic HTML) and CSS (Cascading Style Sheets) are often used together to create dynamic, interactive, and visually appealing web pages.

DHTML CSS provides:

1. Interactivity:

- DHTML enables interactivity through JavaScript, allowing you to manipulate HTML elements on the page dynamically. You can change the content, style, and behavior of elements based on user actions (e.g., mouse clicks, hovers, form submissions).
- CSS can be used to style elements that are dynamically modified by DHTML, ensuring a consistent and visually pleasing appearance.

2. Animation and Effects:

- DHTML allows you to create animations and special effects like fading, sliding, and moving elements on the page.
- CSS can be utilized to enhance these effects, providing transitions and keyframe animations for smoother and more visually appealing results.

3. Event Handling:

- DHTML enables you to attach event handlers to HTML elements to respond to user interactions.
- CSS pseudo-classes (e.g., `:hover`, `:focus`) can be used to style elements when they are interacted with, giving visual feedback to users.

4. Dynamic Content Updates:

- DHTML allows you to dynamically load new content into a web page without refreshing the entire page.
- CSS styles can be applied to the newly loaded content, ensuring it fits seamlessly into the overall design.

5. Layout and Design:

- CSS is the primary tool for defining the layout and visual design of a web page, controlling elements' positioning, size, colors, fonts, and more.

- DHTML can be used to modify the layout and styles dynamically, adjusting the appearance of elements based on user actions or other events.

6. Responsiveness:

- CSS is crucial for creating responsive web design, ensuring that web pages adapt to different screen sizes and devices.
- DHTML can complement this by providing interactive elements that adjust their behavior based on the user's device or screen size.

7. Web Page Enhancements:

- Combining DHTML and CSS allows you to create various enhancements like image sliders, modal dialogs, tooltips, and interactive forms.